

# COMP1531

## Software Engineering Fundamentals

Week 1  
Software Processes

# Why are we here?

— — —

- Fundamentals of software engineering

# Why are We Here?

---

- “Super relevant to skills needed in the workplace. I am currently doing side projects building web apps and this course has helped me significantly in my journey. It is applicable to the real world.”
  - 25T3 Student
- “I really enjoyed being able to finally create a tangible product at the end and have something to show for it, it was really nice to see the applications of software engineering in the real world.”
  - 25T3 Student

# Why are We Here?

— — —

- “The course did a great job of simulating real-world software engineering practices. I liked how it focused on teamwork, GitLab workflows, CI/CD pipelines, and backend development using TypeScript and Express. The project gave me hands-on experience building and testing APIs, which helped me understand version control, code reviews, and collaboration. The tutors and lab structure were also helpful, especially when explaining debugging and testing strategies. Overall, it felt like a good introduction to professional engineering processes rather than just coding.”
  - 25T3 student



# What is Software Engineering?

— — —

- What's the difference between Computer Science and Software Engineering?



# Computer Science vs Software Engineering

— — —

- Computer science
  - Understanding how computers work and function
  - Understanding how we make computers do things
  - What can computers do, and how efficiently?
- Software engineering
  - Focus on building software for people
  - Building the right software with the right practices
  - Maintainable over time
  - Software engineering is an extension of Computer Science
- You can engage in both computer science and software engineering at the same time

# Computer Science vs Software Engineering

— — —

- “Collaboration is a key difference between computer science and software engineering”
  - Robert Pike



# Why is Software Engineering Important?

---

- Have you tried to fix a bug only to break something?
- How do you work in massive teams effectively?
- Why do apps like Instagram and Tiktok get updated regularly?
- What happens if a banking app or a self-driving car misinterprets that data?



# From Small Projects to Big Impacts

---

- Student code
  - It works...probably...we think...
  - Bugs don't matter after submission
  - No one is touching it again
- Industry code
  - What if there are millions of users?
  - How many devices must we support?
  - What if something goes wrong?

```
private bool IsEven(int number){  
    if (number == 1) return false;  
    else if (number == 2) return true;  
    else if (number == 3) return false;  
    else if (number == 4) return true;  
    else if (number == 5) return false;  
    else if (number == 6) return true;  
    else if (number == 7) return false;  
    else if (number == 8) return true;  
    else if (number == 9) return false;  
    else if (number == 10) return true;  
    else if (number == 11) return false;  
    else if (number == 12) return true;  
    else if (number == 13) return false;  
    else if (number == 14) return true;  
    else if (number == 15) return false;  
    else if (number == 16) return true;  
    else if (number == 17) return false;  
    else if (number == 18) return true;  
    else if (number == 19) return false;  
    else if (number == 20) return true;  
    else if (number == 21) return false;  
}
```

# Key Takeaways

— — —

- Software Engineering is more than just coding
  - It's more than functionality
  - Scalable
  - Maintainable
  - Secure
- Poor software engineering can cost:
  - Money
  - Lives

# Understanding Software Process

Software Development Life Cycle (SDLC) is a structured approach to software development that includes:



# Why is a structured process important?

— — —

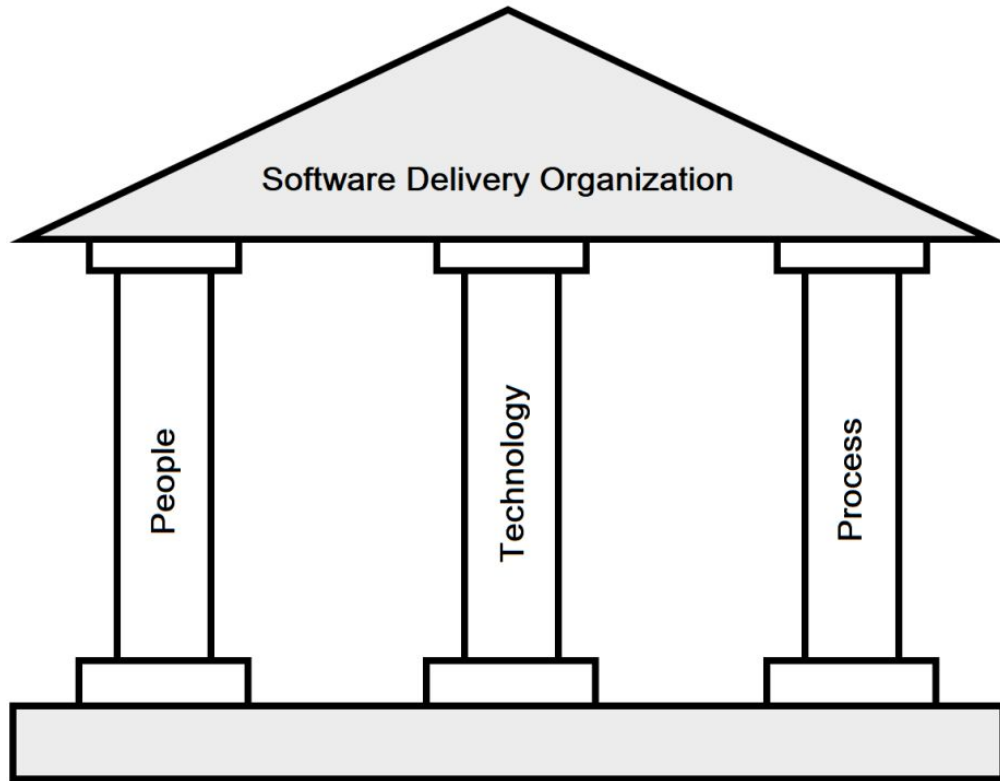
- We're building things that matter
  - Imagine just YOLOing a house
- We need
  - Efficiency
    - Work is done in a logical order
  - Predictability
    - Teams know what to do, and when
  - Quality Assurance
    - Fewer errors, more maintainable a structured process



Eg. for building a house: you need a plan, materials, workers, and inspections to ensure a strong structure.)

# The 3 Pillars of Software Quality and Productivity

---



# The 3 Pillars of Software Quality and Productivity

— — —

1. **People** (Developers & Teams) - Software is built by people, and their skills & teamwork affect the outcome.
2. **Process** (Structured Development Approach) - A well-defined process ensures efficiency, reduces errors, and maintains quality.
3. **Technology** (Tools & Automation) - The right development tools, testing frameworks, and automation improve productivity.

# Common Software Development Models

---

- Waterfall
  - Linear and structured
- Prototyping
  - Early feedback
- Iterative Development
  - Continuous improvement
- Agile
  - Rapid, collaborative
- ...and more!



# The Waterfall Model (Traditional Approach)

— — —

1. Requirements gathering
  - Understanding the clients needs
2. System design
  - Creating the architecture and technical plan
3. Implementation
  - Writing the actual code based on design
4. Testing
  - Check if the software works
5. Deployment
  - Release the software for real users
6. Maintenance
  - Fixing bugs, updating, adapting



# Important Notes

— — —

- In this course, we will be working in groups on a software engineering project (over 9 weeks).
- We can only help you if you communicate with us
  - Is your group bad?
    - TELL US
  - Is there a problem with your assessment iteration/marks?
    - TELL US
      - Within 7 days of the marks release
- We WANT to help you!
  - But we can't if you don't tell us what's wrong
- We are a part of your team, and we want you to succeed
  - We will do everything we can to help you succeed, but we can only do it if you work with us

# Feedback

— — —



Or go to the [form here](#)